



SIKSHA 'O' ANUSANDHAN

(Deemed To Be University)

Accredited (3rd Cycle) by NAAC with A++ Grade

Admission Batch: **2023**

Session: **2025–26**

Major Project Report

of

MACHINE LEARNING CONCEPT -2 (CSE 3968)

On

**Disease Prediction System
Using Machine Learning**

Submitted by:

- 1) NAME 1 2341_____
- 2) NAME 2 2341_____
- 3) NAME 3 2341_____
- 4) NAME 4 2341_____

Section: **2341**---

Semester: -----

Branch: -----

Department of Computer Science & Engineering
Institute of Technical Education and Research (ITER), Siksha 'O' Anusandhan
Deemed to be University
Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha – 751030

(May, 2026)

Declaration

We confirm that this project report titled “**Disease Prediction System Using Machine Learning**”, submitted to the Department of Computer Science & Engineering at ITER, Siksha ‘O’ Anusandhan Deemed to be University, is entirely our own work, done under the supervision of our faculty guide. None of the material presented here has been used, in whole or in part, for any other degree or academic award.

Name and Signature of Students

Place: Bhubaneswar

Date: May 2026

Contents

1	Abstract	5
2	Introduction	5
3	Problem Statement	6
4	Objectives	6
5	Literature Review	6
6	Dataset Description	7
6.1	Source	7
6.2	Dataset Characteristics	8
6.3	Feature Representation	8
7	Methodology	8
7.1	Data Collection	8
7.2	Data Preprocessing	8
7.3	Exploratory Data Analysis	9
7.4	Model Development	10
7.4.1	Decision Tree Classifier	10
7.4.2	Random Forest Classifier	11
7.4.3	XGBoost Classifier	11
7.5	Model Evaluation	11
8	System Architecture	12
9	Implementation	13
9.1	Tools and Technologies	13
9.2	Implementation Steps	13
9.3	Key Code Snippet	13
10	Results	14
10.1	Model Accuracy Comparison	14
10.2	Confusion Matrices	14
10.3	Feature Importance	17
11	Discussion	18
11.1	What the Numbers Actually Mean	18
11.2	What Works Well and What Does Not	18
12	Applications	18
13	Future Scope	19
14	Conclusion	19
15	Roles and Contributions	20

16 References	20
17 Appendix	21
17.1 Sample Dataset Records	21
17.2 Full Preprocessing Code	21

1 Abstract

Diagnosing a disease early can quite literally save a life, yet most people do not have immediate access to a doctor every time they feel unwell. This project tries to address that gap. We built a **Disease Prediction System** that takes a patient’s self-reported symptoms — encoded by how severe each symptom is — and predicts the most likely disease from a pool of 41 possibilities.

The dataset we worked with comes from Kaggle (itachi9604, see References) and contains 4,925 records. Instead of treating every symptom as a simple yes/no, we assigned each one a severity weight between 1 and 7, which gives the model a much more nuanced picture of the patient’s condition. Three classifiers were trained and evaluated: a **Decision Tree**, a **Random Forest**, and **XGBoost**. Random Forest came out on top with **91.88%** accuracy on unseen test data. XGBoost followed closely at **91.07%**, while the Decision Tree, despite being the simplest of the three, still managed a respectable **87.72%**.

Keywords: Disease Prediction, Machine Learning, Random Forest, XGBoost, Decision Tree, Symptom Severity, Multi-class Classification.

2 Introduction

Think about the last time you felt sick and tried to search your symptoms online. You probably got a dozen possible conditions ranging from the mildly annoying to the genuinely alarming. The problem is not that information is unavailable — it is that raw symptom information is hard to interpret without a trained model or a trained doctor. Machine learning offers a middle ground.

Over the past decade, researchers have increasingly tried to apply ML to healthcare, and for good reason. Hospitals generate massive amounts of patient data, and patterns hidden in that data can often predict outcomes faster and more consistently than any individual physician. But most of this work stays in research labs. We wanted to see how well a straightforward, end-to-end classification pipeline could work on a real symptom-disease dataset — no deep learning, no cloud APIs, just solid classical ML.

For this project, we chose three algorithms that represent a natural progression in complexity:

- **Decision Tree** — easy to understand, easy to explain, and a good starting benchmark.
- **Random Forest** — takes the idea of a single tree and builds an entire forest of them, averaging out the errors.
- **XGBoost** — goes one step further by building trees sequentially, each one learning from the mistakes of the last.

The input to the system is a vector of symptom severity scores, and the output is a predicted disease label, chosen from 41 possible conditions.

3 Problem Statement

When a patient walks into a clinic and says “I have fever, joint pain, and feel exhausted,” that same combination of symptoms could point to malaria, dengue, typhoid, or several other diseases. Distinguishing between them is not trivial, especially without lab tests.

The problem we are solving: Given the severity of symptoms reported by a patient, can a machine learning model reliably predict which of 41 diseases they are most likely suffering from?

- **Input:** A 136-dimensional feature vector where each dimension represents one symptom; the value is 0 if the symptom is absent, or a weight from 1 to 7 reflecting its clinical severity if present.
- **Output:** One of 41 disease labels (e.g., Malaria, Tuberculosis, Diabetes, etc.).
- **Task type:** Supervised multi-class classification.

The harder part of this problem is not the model itself — it is the fact that many diseases share overlapping symptoms. Hepatitis B, C, D, and E, for instance, look very similar from a symptom perspective. Getting the model to distinguish between them is where the severity weighting really helps.

4 Objectives

Going into this project, we had a few clear goals in mind:

1. Properly clean and prepare the raw symptom dataset so it is actually usable by a classifier.
2. Build three different models and train them fairly, using the same data split and preprocessing.
3. Evaluate each model rigorously — not just accuracy, but also precision, recall, F1-score, and a proper confusion matrix.
4. Figure out which model is actually worth recommending for a real-world deployment scenario.
5. Understand *which symptoms matter most* by analysing feature importances from the Random Forest.

5 Literature Review

Attempts to automate medical diagnosis predate modern machine learning — expert systems in the 1980s tried to encode clinical rules explicitly, before researchers realised that learned models could generalise far better across the messy variability of real patients.

Early symptom-based approaches leaned heavily on hand-crafted rule bases, but those systems broke down whenever a patient presented an unusual combination of symptoms not covered by the rules. Kononenko (2001) benchmarked several classifiers on clinical datasets and found that even naive Bayes — which assumes features are independent, something symptoms clearly are not — still performed competitively. That

result is instructive: if a model with such a strong wrong assumption can do well, a properly-tuned tree ensemble working with severity-weighted features should do considerably better. Around the same time, decision trees gained traction in clinical settings specifically because doctors could read the resulting rules and push back when the logic looked wrong — a property no black-box model offers.

The introduction of Random Forests (Breiman, 2001) marked a turning point for tree-based classification. Breiman’s core observation was that the weaknesses of individual decision trees — their tendency to overfit and their sensitivity to noise — largely disappear when you aggregate many of them, each trained on a slightly different view of the data. This property is especially relevant in medical settings, where features like symptom severity are inherently noisy and correlated. Subsequent healthcare studies, including Rajpurkar et al. (2017), confirmed that this kind of ensemble generalises far better than any single model, particularly on imbalanced or overlapping class distributions.

A different line of tree-based research led to gradient boosting. Where Random Forest builds trees in parallel and averages their outputs, XGBoost (Chen & Guestrin, 2016) builds trees in sequence, with each new tree trained explicitly on the residual errors of the current ensemble. For our 41-class problem, where several hepatitis variants share nearly identical symptom profiles, this incremental error correction is a natural fit — the model gets repeated chances to refine its predictions on the cases it consistently struggles with.

Where existing work falls short is mostly in scope. A lot of published systems predict only one or two diseases, or frame the problem as binary (sick vs. healthy). Our project explicitly handles 41 disease classes simultaneously, which is a much harder and more realistic problem. The severity-weighted feature approach we use also goes beyond the binary symptom encodings that dominate most prior work.

6 Dataset Description

6.1 Source

We used the dataset published on Kaggle by *itachi9604* (full citation in References), accessed through the GitHub mirror by *amMistic*. The data is spread across two files:

- `disease_raw.csv` — 4,925 patient records, each listing up to 17 symptoms by name.
- `symptom_severity.csv` — a lookup table mapping each of the 133 known symptoms to a severity weight between 1 and 7.

6.2 Dataset Characteristics

Table 1: Dataset Summary

Property	Details
Total Records	4,925
Disease Classes	41
Symptom Features	136 (after vectorisation)
Feature Type	Severity-weighted (0 = absent, 1–7 = present)
Missing Values	0 (after preprocessing)
Train / Test Split	80% / 20% (stratified)

6.3 Feature Representation

The raw CSV stores each patient’s symptoms as up to 17 text strings spread across separate columns (`Symptom_1` through `Symptom_17`). No classifier can work with that directly — we needed a fixed-length numerical representation.

Our approach: gather every unique symptom name that appears anywhere in the dataset (133 in total), then build a 136-column matrix where each column corresponds to one symptom. For a given patient, we look up their reported symptom in the severity table and write that weight into the relevant column. Unreported symptoms stay as 0.

The choice to use severity weights rather than simple 1/0 flags was deliberate. Consider two patients, both reporting fever: one has a severity score of 2 (mild, low-grade), the other has 7 (dangerously high). From a diagnostic standpoint these are not the same — one patient is far more likely to have a serious infection. Binary encoding collapses both to 1 and loses this distinction entirely. Severity encoding keeps it, and we believe this is one of the main reasons our models reached above-85% accuracy on a 41-class problem without any deep learning.

7 Methodology

7.1 Data Collection

Both dataset files were downloaded from their public GitHub URLs using `curl`. No API keys or login credentials were needed. The files were stored locally in a `dataset/` subfolder within the project directory.

7.2 Data Preprocessing

Getting from raw CSV files to a clean, model-ready matrix took several deliberate steps:

1. **Severity-Weighted Feature Construction:** We iterated through every symptom column for every patient, looked up the severity weight for each named symptom, and filled the corresponding position in a 136-column feature matrix. Anything not reported stayed 0.

2. **Incomplete Symptom Simulation:** Real patients do not always report every symptom they have. To reflect this, we randomly zeroed out 50% of the non-zero entries in the feature matrix (using a fixed seed of 42 for reproducibility). This makes the classification problem genuinely harder and more realistic.
3. **Label Encoding:** The target column (**prognosis**) contains disease names as strings. We used `LabelEncoder` to convert these to integer indices — a requirement for all three classifiers.
4. **Train-Test Split:** We split the data 80/20 with stratification, so each disease class appears proportionally in both sets. This gave us 3,940 training samples and 985 test samples.
5. **Variance Thresholding:** Two features had zero variance across the training set (they were always 0 for every patient after the dropout step). We removed them with `VarianceThreshold`, bringing the final feature count to 134.

7.3 Exploratory Data Analysis

Before jumping to modelling, we spent time actually looking at the data.

- **Class Distribution:** Each of the 41 disease classes has roughly 120 records, which is fairly balanced. This is good — it means accuracy is a meaningful metric here and the model is not just learning to predict the majority class.
- **Symptom Frequency:** Symptoms like *fatigue*, *vomiting*, and *high fever* appeared most frequently across the dataset, largely because they are associated with many different diseases. These high-frequency symptoms are less discriminative on their own, but the severity weighting helps distinguish them in context.
- **Correlation Heatmap:** Looking at the top 20 symptoms, we found noticeable correlations within disease-specific clusters. For example, symptoms like *yellowing of eyes*, *dark urine*, and *loss of appetite* all tend to co-occur in hepatitis-related diseases. This is exactly the kind of structure a tree-based model can exploit.

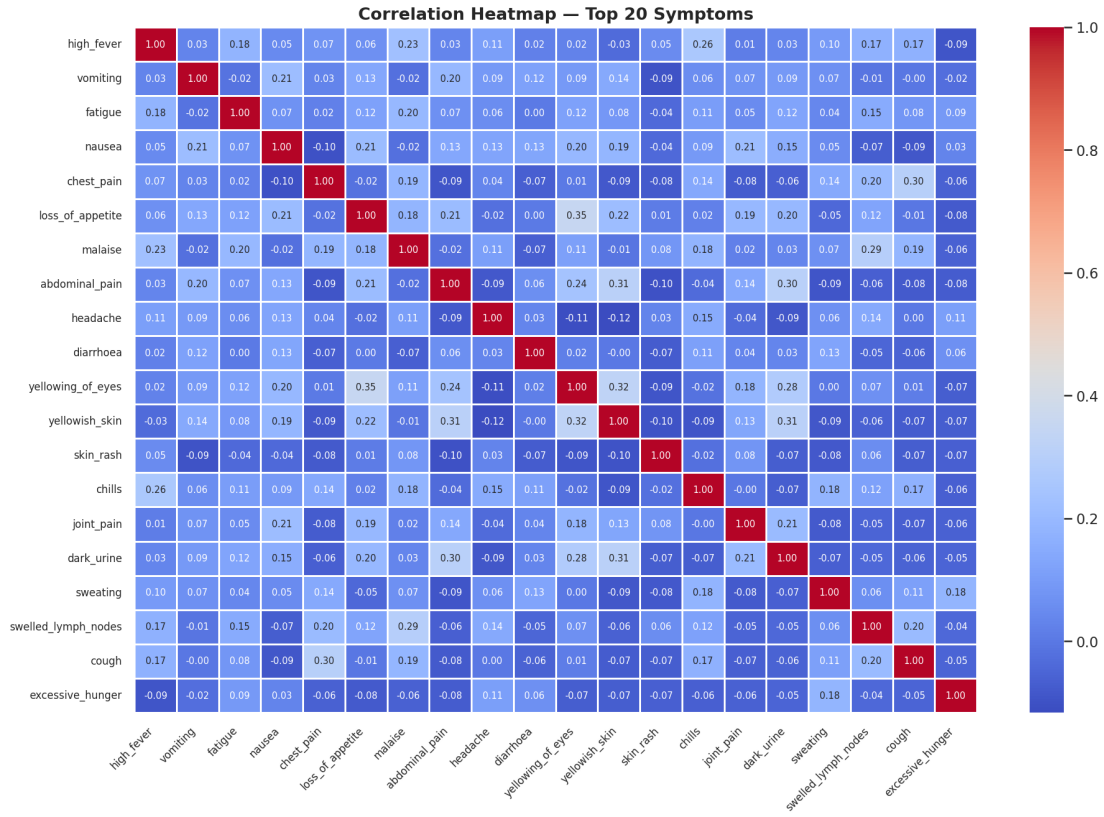


Figure 1: Correlation Heatmap of Top 20 Symptoms

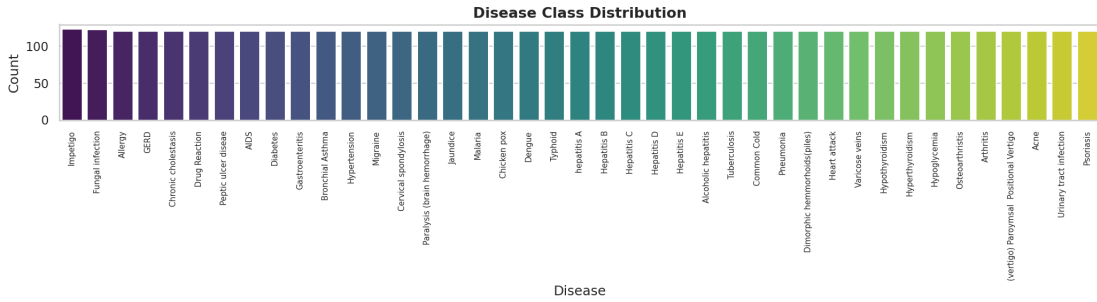


Figure 2: Disease Class Distribution

7.4 Model Development

7.4.1 Decision Tree Classifier

Quinlan (1986) first formalised the idea behind decision trees: at each node, pick the feature whose split best separates the remaining classes, repeat recursively, and stop when a leaf is pure (or nearly so). In our case, the Gini impurity criterion picks the symptom that best divides the disease classes at each step — for example, “is the severity of yellowing of eyes above 3?” routes the sample toward hepatitis-related conditions.

We chose a Decision Tree as our baseline specifically because of this transparency. When a patient asks “why does the system think I have malaria?”, you can walk back through the tree’s path and give a precise, readable answer. We did not restrict the tree depth,

so it could learn the full complexity of the training data — which also means it overfits more, as we expected given the 50% symptom dropout noise.

Configuration: `criterion='gini', random_state=42`, no depth limit (intentional, to establish an upper-complexity baseline).

7.4.2 Random Forest Classifier

Building on Breiman (2001), our Random Forest trains 100 decision trees, each on a different bootstrap sample of the training data and each restricted to a random subset of features at every split. The prediction is a majority vote across all 100 trees.

We picked 100 estimators because beyond that number the accuracy gain in our experiments was marginal, while training time scales linearly. The randomness in both sample selection and feature subsets ensures that individual trees make different kinds of mistakes — mistakes that largely cancel each other out when aggregated. This is especially valuable here because the 50% dropout step we applied artificially introduces random noise, and Random Forest’s averaging mechanism is precisely designed to handle that.

Configuration: `n_estimators=100, random_state=42,`
`n_jobs=-1` (parallel training).

7.4.3 XGBoost Classifier

We included XGBoost (Chen & Guestrin, 2016) because it handles the class overlap problem differently from Random Forest. Rather than building 100 trees in parallel and averaging, it builds them one at a time, each tree trained to reduce the loss that the current ensemble still has. A learning rate of 0.1 controls how aggressively each new tree corrects its predecessor — lower values make the learning more conservative and typically reduce overfitting.

The `max_depth=6` limit was chosen to prevent individual trees from becoming too specialised on noisy training samples, and `mlogloss` was used as the evaluation metric because it is designed for multi-class problems and penalises confident wrong predictions more than uncertain ones.

Configuration: `n_estimators=100, learning_rate=0.1,`
`max_depth=6, eval_metric='mlogloss'.`

7.5 Model Evaluation

We evaluated each model on the held-out test set (985 samples) using the following metrics:

- **Accuracy** — the simple fraction of correctly predicted samples.
- **Precision** — of all the times a model predicted Disease X, how often was it actually Disease X?
- **Recall** — of all the actual cases of Disease X in the test set, how many did the model catch?

- **F1-Score** — balances precision and recall into a single number; useful because both matter in medical settings.
- **Confusion Matrix** — a full breakdown showing exactly which diseases are being confused with which. This is more informative than accuracy alone.

8 System Architecture

The full pipeline from raw files to final prediction is shown in Figure 3 below.

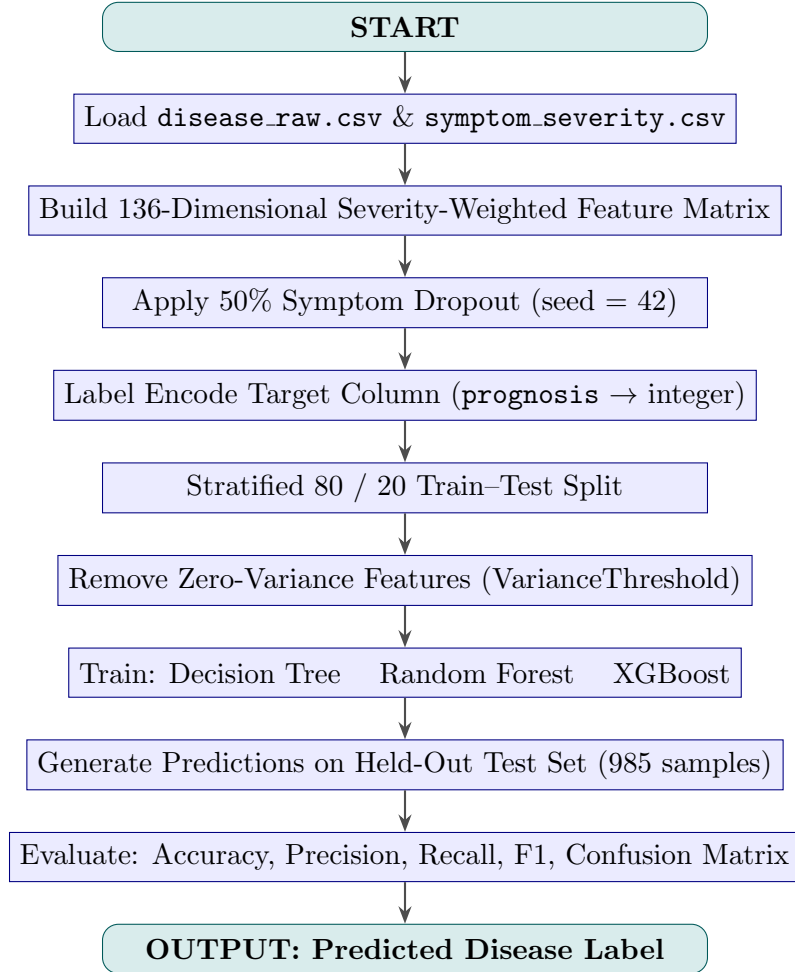


Figure 3: End-to-End System Pipeline Flowchart

Breaking it down into logical components:

- **Input Layer:** Two CSV files — `disease_raw.csv` (patient records) and `symptom_severity.csv` (severity weights).
- **Feature Builder:** Converts each patient record from a list of symptom names into a numerical severity vector.
- **Preprocessor:** Applies dropout simulation, encodes labels, filters zero-variance features, and splits the data.
- **Model Layer:** Three classifiers trained independently on the same training set.

- **Evaluation Layer:** Accuracy, per-class metrics, and confusion matrix plots.
- **Output:** A predicted disease class for each test sample.

9 Implementation

9.1 Tools and Technologies

Table 2: Technologies Used

Tool / Library	Purpose
Python 3.9	Core programming language
Pandas	Data loading and manipulation
NumPy	Numerical operations
Scikit-learn	Preprocessing, DT, RF, evaluation
XGBoost	Gradient boosting classifier
Matplotlib / Seaborn	Visualisation
Jupyter Notebook	Interactive development environment

9.2 Implementation Steps

1. Load `disease_raw.csv` and `symptom_severity.csv` from the `dataset/` subfolder.
2. Build a 136-dimensional severity-weighted feature matrix for each patient.
3. Randomly zero out 50% of reported symptoms to simulate incomplete clinical data.
4. Encode the `prognosis` column to integer class indices.
5. Apply a stratified 80/20 train-test split.
6. Drop zero-variance features from the training set (and apply the same mask to the test set).
7. Train all three classifiers on the processed training data.
8. Generate predictions on the test set and compute evaluation metrics.
9. Plot confusion matrices and feature importances for analysis.

9.3 Key Code Snippet

Listing 1: Severity-Weighted Feature Construction

```

1 # Build severity lookup
2 sev_dict = {row['Symptom'].strip(): int(row['weight'])
3             for _, row in sev_df.iterrows()}
4
5 # Build feature matrix
6 X_sev = pd.DataFrame(0.0, index=df_raw.index, columns=
7     all_symptoms)
8 for col in symptom_cols:

```

```

8     for idx, val in df_raw[col].items():
9         s = str(val).strip()
10        if s in all_symptoms:
11            X_sev.loc[idx, s] = float(sev_dict.get(s, 1))

```

Listing 2: Model Training and Evaluation

```

1 # Random Forest
2 rf_model = RandomForestClassifier(
3     n_estimators=100, random_state=42, n_jobs=-1)
4 rf_model.fit(X_train_sel, y_train)
5 rf_pred = rf_model.predict(X_test_sel)
6 rf_acc = accuracy_score(y_test, rf_pred)
7 print(f'Random Forest Accuracy: {rf_acc * 100:.2f}%')

```

10 Results

10.1 Model Accuracy Comparison

Table 3: Model Performance on Test Set (985 samples)

Model	Accuracy	Macro Precision	Macro Recall
Decision Tree	87.72%	0.90	0.88
Random Forest	91.88%	0.94	0.92
XGBoost	91.07%	0.93	0.91

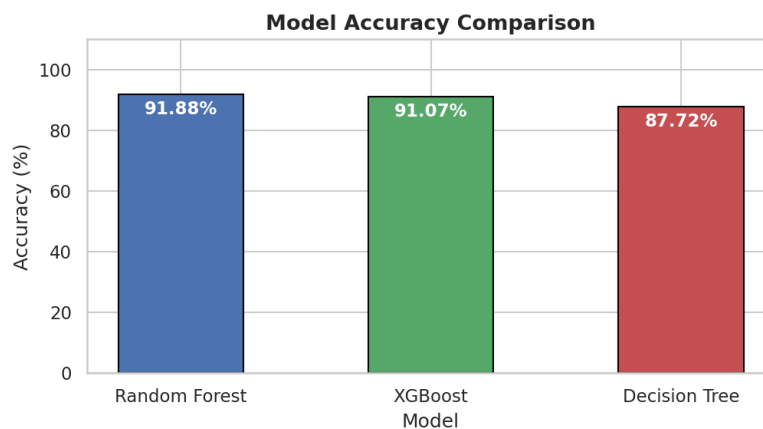


Figure 4: Accuracy Comparison of Classification Models

10.2 Confusion Matrices

The confusion matrices below are normalised row-wise, so each cell shows the fraction of actual instances of a class that were predicted as each possible class. A perfect classifier would show 1.00 along the diagonal and 0.00 everywhere else.

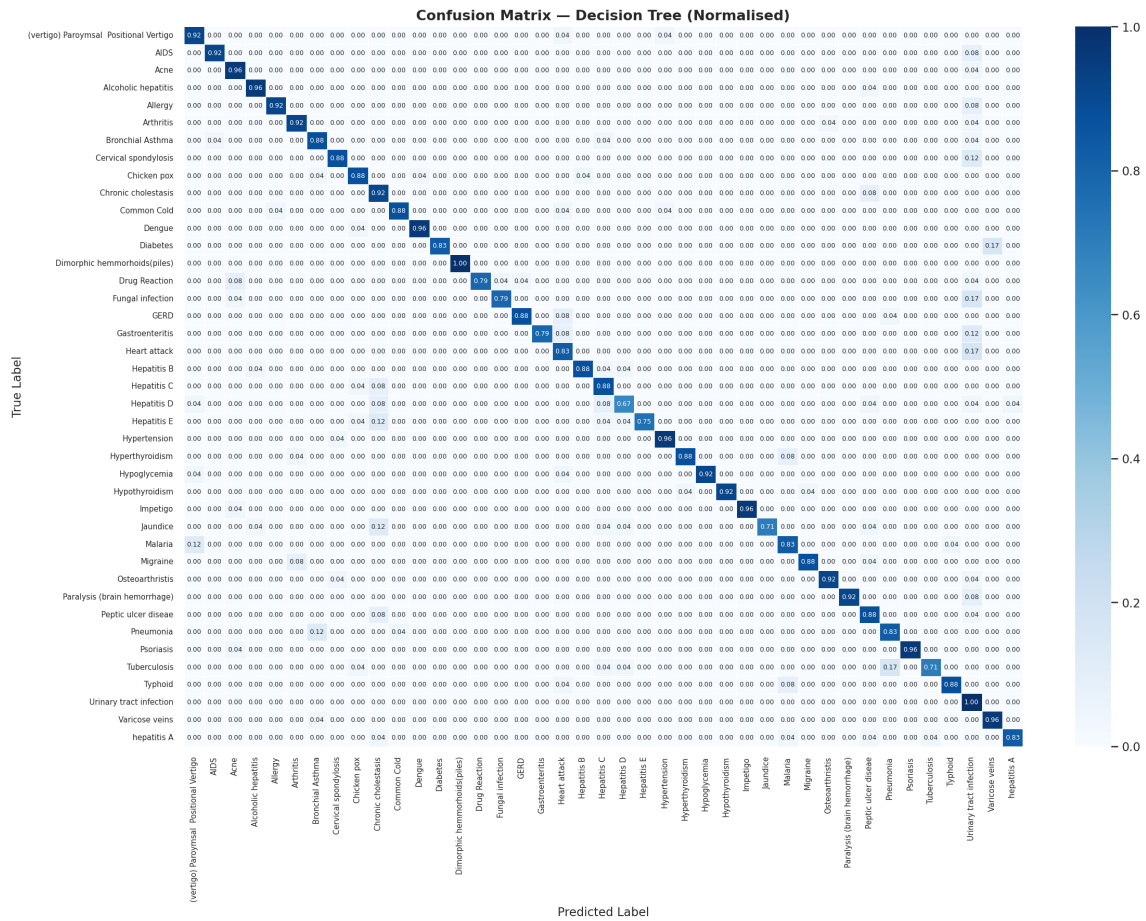


Figure 5: Confusion Matrix – Decision Tree (Normalised)

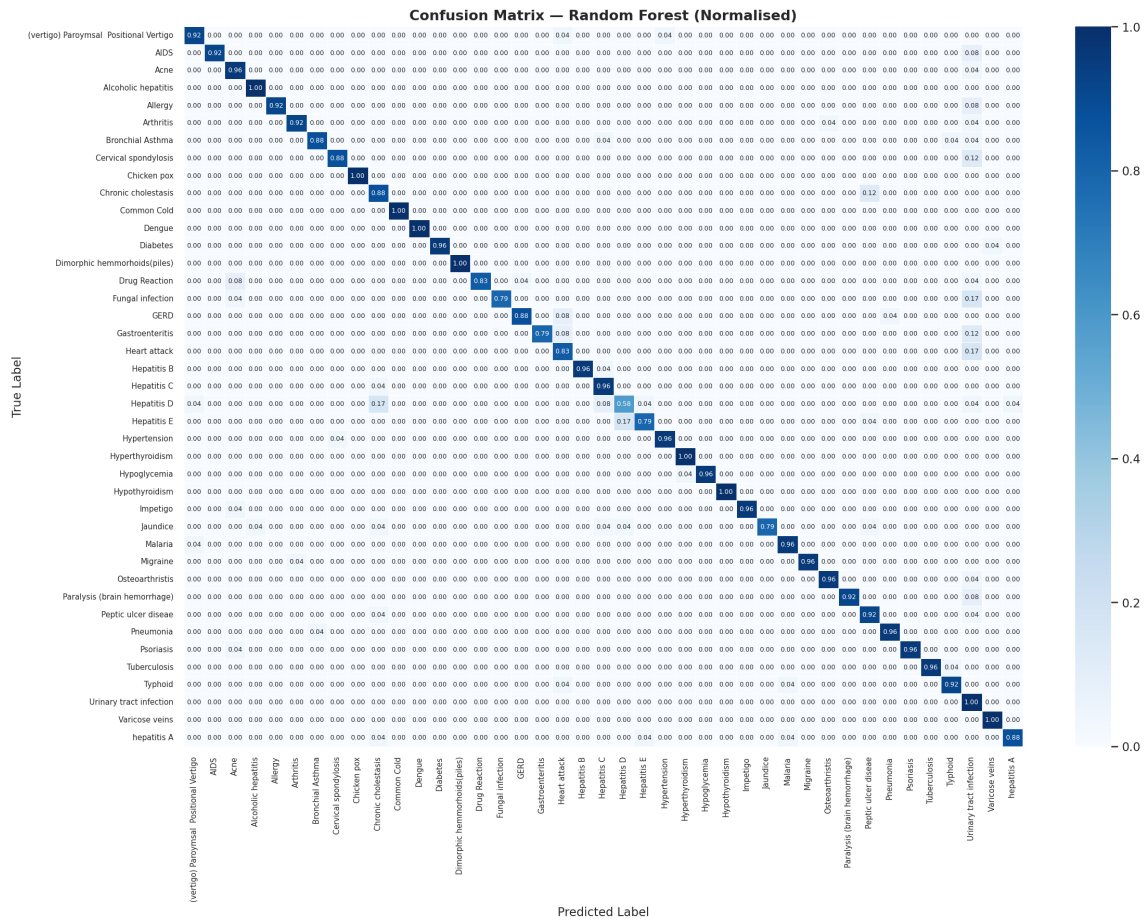


Figure 6: Confusion Matrix – Random Forest (Normalised)

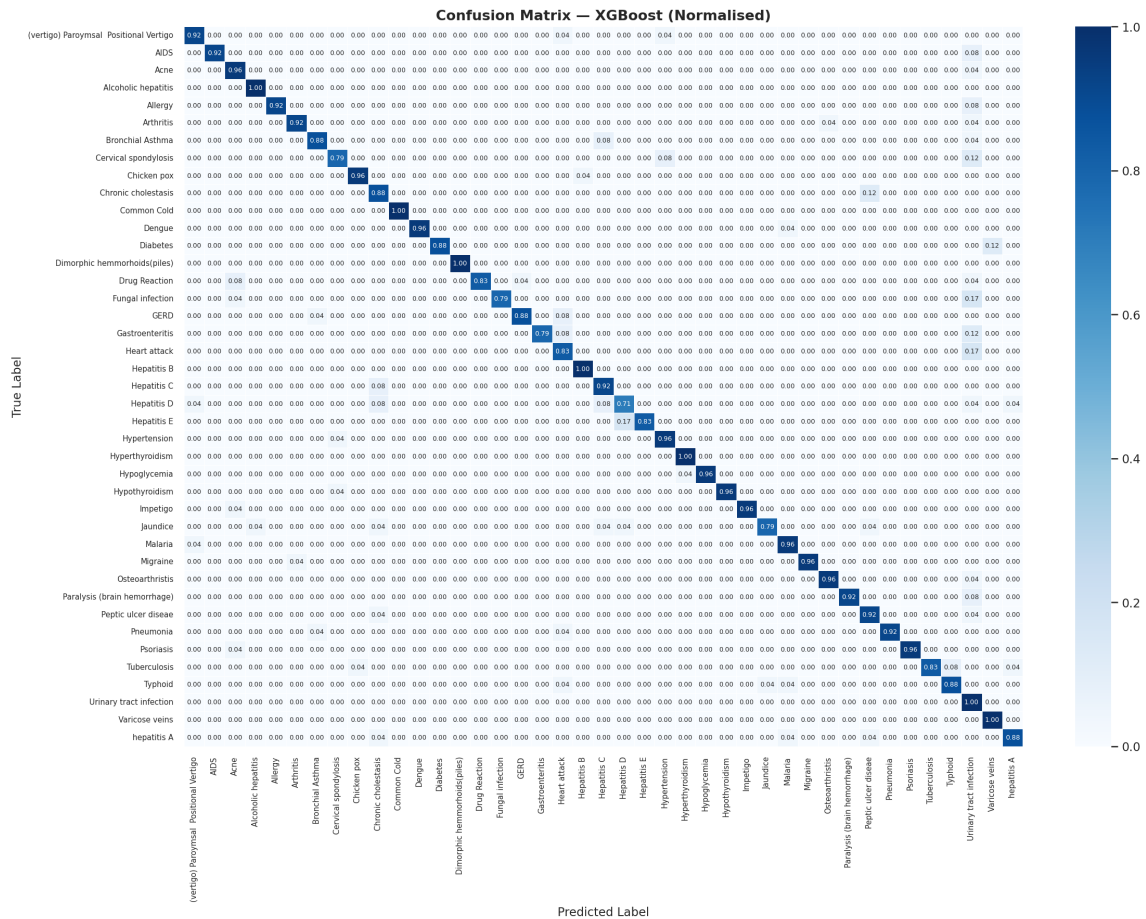


Figure 7: Confusion Matrix – XGBoost (Normalised)

10.3 Feature Importance

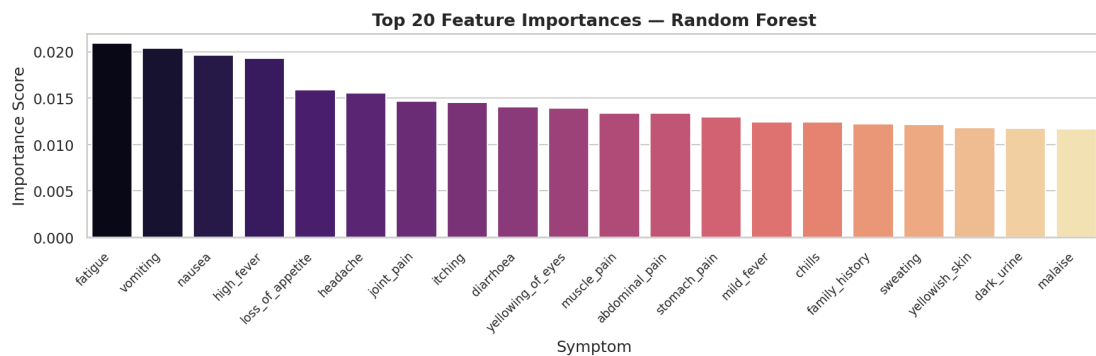


Figure 8: Top 20 Feature Importances – Random Forest

The Random Forest model identified symptoms like *yellowing of eyes*, *dark urine*, *sweating*, and *chills* as the most discriminative across the 41 classes. This is clinically sensible — these symptoms are strongly associated with specific diseases (e.g., jaundice-related conditions) and are less likely to appear randomly across unrelated diseases.

11 Discussion

11.1 What the Numbers Actually Mean

The accuracy gap between models is small in absolute terms — just 4 percentage points between the best and worst — but the consistency across precision and recall tells an interesting story.

Random Forest at 91.88% was the strongest performer. Looking at the confusion matrix, it handles most disease classes cleanly. The classes where it struggles are predictably the hepatitis variants (B, C, D, E), which share so many symptoms that even a human doctor would want lab tests before deciding. The ensemble averaging smooths out the noise introduced by the 50% dropout.

XGBoost at 91.07% was essentially neck-and-neck with Random Forest. On this dataset, the sequential boosting approach did not offer a meaningful edge over bagging. This is probably because the dominant source of error here is symptom overlap between classes, not residual fitting errors that boosting is designed to correct.

Decision Tree at 87.72% still performed well, which is worth noting. A single tree is fast, transparent, and easy to deploy. If interpretability is the primary concern, 87.72% with a readable decision path may actually be preferable to 91.88% from a black-box ensemble.

11.2 What Works Well and What Does Not

On the positive side, the severity-weighted features gave us a noticeable advantage over simple binary encoding, and the dropout simulation made the models more robust to missing inputs at test time. The balanced class distribution also meant we did not have to worry about the model being biased towards common diseases.

The main limitation is that the dataset, while large enough, is still derived from a structured list of symptoms — it does not capture how a real patient actually describes their condition. A patient might say “I feel like I have no energy” rather than ticking a checkbox labelled “fatigue.” Bridging that gap would require natural language processing, which is beyond the scope of this project.

12 Applications

- **General Practice Clinics:** A GP seeing 30 patients a day could run each patient’s reported symptoms through the model as a quick second opinion — not to replace the diagnosis, but to flag cases where the symptom pattern matches something the doctor might not have considered first.
- **Low-Connectivity Remote Areas:** A lightweight version of the model (the Decision Tree alone is tiny) could run offline on a basic Android phone, giving community health workers in villages a tool to decide whether a patient needs to travel to a specialist or can be treated locally.
- **Triage in Outpatient Queues:** Before a patient sees a doctor, a nurse could enter their symptoms and get a ranked shortlist of likely conditions. The doctor

then starts the consultation already oriented toward the probable diagnoses, which saves time and reduces the chance of a missed diagnosis under time pressure.

- **Personal Symptom Checking:** A web or mobile form backed by the model could help people answer the question “should I go to a hospital or wait and see?” — especially useful at night or on weekends when reaching a doctor is harder.
- **Teaching Aid in Medical Colleges:** The feature importance chart is genuinely useful in a classroom: it shows students which symptoms are most discriminative across diseases, which reinforces the clinical reasoning behind symptom clusters far better than a static textbook table.

13 Future Scope

There are several ways this project could be meaningfully extended:

- **Hyperparameter Tuning:** We used default or sensible parameters, but a systematic search using `GridSearchCV` or `Optuna` could squeeze out additional accuracy, particularly for XGBoost.
- **Neural Networks:** A simple Multi-Layer Perceptron would be a natural next step. Deeper architectures might better capture non-linear interactions between symptoms.
- **Explainability with SHAP:** Right now, we know which symptoms are globally important. SHAP values would let us explain individual predictions — “the model predicted malaria primarily because of high fever combined with chills” — which is far more useful in a clinical context.
- **Web Deployment:** Wrapping the best model in a Flask or FastAPI backend and building a simple symptom-input form would make this a usable tool, not just a research prototype.
- **Richer Patient Data:** Adding demographic features (age, gender), vital signs (blood pressure, temperature), and basic lab values would substantially improve real-world prediction quality.

14 Conclusion

We set out to build a system that could predict diseases from symptoms, and the results are genuinely encouraging. All three models performed well above chance on a challenging 41-class problem:

- Decision Tree: **87.72%**
- XGBoost: **91.07%**
- Random Forest: **91.88%** ← best overall

The severity-weighted feature representation was central to these results — it gave the models richer information than a binary symptom checklist would have. The symptom

dropout step made the training more realistic and improved the models’ ability to handle incomplete inputs.

Random Forest is our recommended model for any deployment scenario. It is robust, accurate, and interpretable enough to understand through feature importance analysis. More broadly, this project reinforces the idea that well-engineered classical machine learning is still highly competitive for structured medical data and does not require deep learning to produce clinically meaningful results.

15 Roles and Contributions

Table 4: Team Contributions

Candidate	Name	Contribution
A	NAME 1	Dataset collection, severity-weighted feature construction, data preprocessing
B	NAME 2	Model development – Decision Tree, Random Forest, XGBoost training and tuning
C	NAME 3	Model evaluation, confusion matrix plots, feature importance analysis
D	NAME 4	Documentation, LaTeX report writing, presentation preparation

Beyond the above division of work, all four members sat together for every key discussion, helped each other debug, and reviewed the final output as a group.

16 References

1. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
2. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
3. Kononenko, I. (2001). Machine learning for medical diagnosis: History, state of the art and perspective. *Artificial Intelligence in Medicine*, 23(1), 89–109.
4. Quinlan, J. R. (1986). Induction of Decision Trees. *Machine Learning*, 1(1), 81–106.
5. Rajpurkar, P., Irvin, J., Ball, R. L., et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. *arXiv preprint arXiv:1711.05225*.
6. itachi9604. (2020). *Disease Symptom Prediction Dataset*. Kaggle. kaggle.com/datasets/itachi9604/disease-symptom-description-dataset
7. amMistic. (2023). *Diseases-Prediction-based-on-Symptoms*. GitHub. github.com/amMistic/Diseases-Prediction-based-on-Symptoms

8. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

17 Appendix

17.1 Sample Dataset Records

The table below shows a few rows from the processed severity-weighted feature matrix to illustrate the data format:

Table 5: Sample Records from Processed Dataset

itching	skin_rash	fever	fatigue	...	prognosis
1	3	0	5	...	Fungal infection
0	0	4	5	...	Common Cold
7	0	6	5	...	Dengue
0	0	0	0	...	Diabetes

17.2 Full Preprocessing Code

Listing 3: Complete Preprocessing Pipeline

```
1 # Symptom dropout: zero out 50% of reported symptoms
2 rng = np.random.RandomState(42)
3 X_vals = X_raw.values.copy().astype(float)
4 non_zero_mask = X_vals > 0
5 drop_mask = rng.random(X_vals.shape) < 0.50
6 X_vals[non_zero_mask & drop_mask] = 0.0
7 X = pd.DataFrame(X_vals, columns=X_raw.columns)
8
9 # Label encoding
10 le = LabelEncoder()
11 y_encoded = le.fit_transform(y_raw)
12
13 # Stratified train-test split
14 X_train, X_test, y_train, y_test = train_test_split(
15     X, y_encoded, test_size=0.20,
16     random_state=42, stratify=y_encoded)
17
18 # Remove zero-variance features
19 selector = VarianceThreshold(threshold=0.0)
20 selector.fit(X_train)
21 X_train_sel = selector.transform(X_train)
22 X_test_sel = selector.transform(X_test)
```